Document Number: P3302R0
Date: 2024-05-21
Audience: SG16
Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2024-03-13 through 2024-05-08

Summaries of SG16 meetings are maintained at https://github.com/sg16-unicode/sg16-meetings. This paper contains a snapshot of select meeting summaries from that repository.

- March 13th, 2024
- April 10th, 2024
- April 24th, 2024
- May 8th, 2024

Previously published SG16 meeting summary papers:

- P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25
- P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20
- P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03
- P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09
- P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22
- P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25
- P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11
- P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27
- P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26
- P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11
- P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24
- P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26
- P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15
- P2605: SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08
- P2678: SG16: Unicode meeting summaries 2022-06-22 through 2022-09-28
- P2766: SG16: Unicode meeting summaries 2022-10-12 through 2022-12-14
- P2891: SG16: Unicode meeting summaries 2023-01-11 through 2023-05-10
- P2995: SG16: Unicode meeting summaries 2023-05-24 through 2023-09-27
- P3174: SG16: Unicode meeting summaries 2023-10-11 through 2024-02-21

March 13th, 2024

Draft agenda:

- P1729R4: Text Parsing.
- P3154R0: Deprecating signed character types in iostreams.

Attendees:

- Alisdair Meredith
- Braden Ganetsky
- Eddie Nolan
- Elias Kosunen
- Fraser Gordon
- Jens Maurer
- Mark de Wever
- Nathan Owens
- Robin Leroy
- Tom Honermann
- Victor Zverovich

Meeting summary:

- A round of introductions was held for new attendee Braden Ganetsky.
- P1729R4: Text Parsing:
  - Elias explained that prior feedback has been addressed and that the paper is expected to be ready for a forwarding poll.
  - Elias reviewed the revision history and the changes requested by SG9.
  - Elias stated that support for stdin will be provided in a future paper; similar to how `std::print()` was proposed after `std::format()` was adopted.
  - Elias proceeded to review each section of the paper.
  - Eddie noted that the comment in the example in section 3.2, "Reading multiple values at once", appears to be missing `values()` following `operator->`.
  - *[Editor's note: The comment appears to be intentional in only referring to `operator->`, but incorrect in stating, "will throw if it doesn't contain a value"; a call to `std::expected<T>::operator->()` exhibits UB if `has_value()` is not true.]*
  - Tom asked, while looking at the example in section 3.4, "Reading multiple values in a loop", if all result values are definitely assigned.
  - Elias explained that the scan result is returned by value and that there is no way to provide an object that is referenced within the result object.
  - Tom asked, while looking at the example in section 3.6, "Scanning a user-defined type", if the use of `std::expected` is required or whether another `std::expected`-like type could be used.
  - Elias replied that a concept-like approach is used in the reference implementation.

- Braden asked if it will be surprising to programmers that `std::scan` reports errors via `std::expected` where as `std::format` uses exceptions.
- Elias responded that a failure to parse input provided at run-time is expected and therefore a different category of error than what is expected when formatting.
- Victor agreed with Elias and stated that this is a reasonable design.
- Mark asked what happens if the scan format string is not valid.
- Elias replied that the format string is constant evaluated and, if not valid, renders the program ill-formed.
- Mark commented that both throwing an exception and returning a `std::expected` value that holds an error type in response to an invalid format string can suffice to produce a compile-time error.
- Elias proceeded to review section 4, "Design".
- Robin requested that, in section 4.2, "Format strings", in the discussion of whitespace, the word "currently" in "Those code points are currently" be struck since the Unicode stability policy ensures these won't change.
- Robin observed that the list of whitespace code points appears to be missing some characters; U+000B LINE TABULATION for example.
- Elias responded that the ASCII line includes a range of code points that includes that character.
- Tom suggested it would be more clear for the list to include all of the `Pattern_White_Space` characters individually.
- Elias continued review in section 4.3.2, "Fill and align", and explained the behavior for scanning of centered text without an explicit width; an unambiguous width cannot be inferred based on surrounding fill characters.
- Tom referenced the `rH` example that scans `"*42**"` with a `"{:*^}"` format specification, noted that the final `*` character is not scanned, and asked for confirmation that the example won't roundtrip with what `std::format()` produces with an explicit field width.
- Elias confirmed.
- Victor suggested double checking how the Python parse project handles that situation.
- Elias responded that he had checked at one point, but would need to do so again.
- *[Editor's note: The "Format Specification" section in the Python parse project description states:*

  > Note that the “center” alignment does not test to make sure the value is centered - it just strips leading and trailing whitespace.

  ]
- Victor pondered whether it is possible to roundtrip in general without field width information and suggested the possibility of not supporting scanning of center aligned text without an explicit field width.
- Elias agreed that such cases could be disallowed.

- Jens questioned whether it might be a good to scale back the options for scanning.
- Jens noted that there are already some asymmetries and provided an example; `std::format()` produces a specific whitespace sequence while `std::scan()` will consume arbitrary whitespace.
- Jens suggested that use of a regular expression to consume fill characters might provide a more practical approach.
- Elias asked if Jens' suggestion is intended just for handling of center alignment or for all field widths.
- Jens clarified that the goal would be for the `r5` example to have a format specifier that consumes an arbitrary number of fill characters.
- Jens stated that perhaps the `r7` example would not be covered by this idea since it has an explicit field width.
- Jens opined that the `r5` example and all those that follow it are a little concerning; particularly with regard to centering.
- Elias responded that section 6.2, "`scanf`-like `[character set]` matching" discusses potential future support for matching regular expressions and discarding characters.
- Elias stated these future directions would cover Jens' suggested approach, but acknowledged that a format specifier option would be convenient.
- Jens stated that full regular expression support would invite complication.
- Mark asked if dynamic field widths are supported.
- Elias replied that they are explicitly disallowed.
- Elias reported that there was a poll in LEWGI that supported compatibility with `std::format` as a guiding principle.
- Elias acknowledged that formatting and scanning are different.
- Jens agreed and stated that compatibility makes sense as long as it makes sense.
- Victor stated that symmetry with `std::format()` is not a goal, but that providing a replacement for `scanf()` is a goal and the motivation for many of these use cases.
- Jens replied that he is not aware of features in `scanf()` that would allow for skipping over fill characters.
- Victor acknowledged the lack of such general features but that the use cases apply when the fill character is a space character.
- Victor asked if iostreams supports skipping fill characters when scanning.
- General uncertainty was expressed.
- Jens reported that it appears that example `r5` cannot be parsed with `scanf()`.
- Tom stated that it sounds like there is some homework to be done.
- Jens suggested that homework be done and that review continue at a future telecon.
- Tom agreed.
- Eddie moved on to section 4.3.3, "Sign, '#', and '0'", and stated that ignoring '+' and '-' signs or leading '0' characters would not be desirable by default, but could be useful in conjunction with the sign and '0' format options.

    - Elias responded that, in his experience, it is more important to have a clean design space than it is to have compatible format strings and that he preferred to not allow those flags in order to avoid confusion.
    - Victor agreed with Elias.
    - Elias explained that there is an additional roundtrip asymmetry when formatted text exceeds an explicit field width; scanning the text with an explicit field width won't consume all of the formatted text.
    - Elias noted that section 4.3.5.2, "Design discussion: Separate flag for thousands separators" will be removed; it was unintentionally left in.
- P3154R0: Deprecating signed character types in iostreams:
    - Elias introduced the paper by explaining that the `signed char` and `unsigned char` inserters and extractors behavior is surprising because those types are treated as character types but are often used as the underlying types of `int8_t` and `uint8_t`.
    - Alisdair asked how `std::format()` handles these types.
    - Elias responded that they are formatted as integer types.
    - Jens suggested updating section 1, "Motivation", to add a `std::format()` example for each of the `std::cout` examples.
    - Alisdair asked about the long term intent and whether these functions might be defined as deleted or specified to have different behavior after a deprecation period.
    - Alisdair asserted that deprecation should be a transitional state; features should not stay deprecated indefinitely.
    - Elias expressed a preference for defining them as deleted due to concerns about just switching to new behavior.
    - Victor expressed strong support for deprecation and stated that these functions are a common source of errors.
    - Victor noted that the existing behavior will remain available but will require an explicit cast to a `char`-based type.
    - Jens stated that a plan to deprecate in C++26, to define these functions as deleted for C++29, and to define them with new behavior for C++40 or so could make sense.
    - Jens expressed strong support for defining these functions as deleted as either a final or further intermediate step.
    - Jens requested gathering some implementation experience by modifying a C++ standard library to define these functions as deleted and then compiling some real world projects to see if any latent bugs are discovered.
    - Jens opined that deprecation is a LEWG concern and that SG16 should offer a recommendation on use of `signed char` and `unsigned char` as character types.
    - Alisdair pondered an option to change the behavior to implementation-defined or unspecified.
    - **Poll 1: Recommend reserving `signed char` and `unsigned char` for use as integer types, not character types.**
        - **Attendees: 11 (1 abstention)**

        - **SF F N A SA**
          7 2 0 1 0
        - **Consensus in favor.**
        - **A: I would like to see the results for the experiment Jens suggested first.**
    - **Poll 2: Forward P3154R0 with the suggested modifications to the motivation section to LEWG for C++26.**
        - **Attendees: 11 (3 abstentions)**
        - **SF F N A SA**
          4 2 1 1 0
        - **Consensus in favor.**
        - **A: The direction is more a matter for LEWG.**
    - Those that abstained from the second poll reported being uneasy with the poll because the proposed change to deprecate these features is not an SG16 concern.
    - Tom explained that his intention with forwarding polls is to confirm that there are no outstanding SG16 concerns that are not either addressed or discussed in the paper; these polls are not intended to state a position on matters that do not fall under SG16's purview.
- Tom reported intent to cancel the scheduled 2024-03-27 SG16 meeting since the WG21 meeting in Tokyo will have just concluded and we'll all be busy catching up with our regular lives.
- Jens expressed support for that cancellation.
- Tom reported that he has historically scheduled SG16 meetings for the 2nd and 4th Wednesday of each month, but that meetings from now through 2024-10-24 were scheduled for every two weeks; whether inadvertently or intentionally with now forgotten intent remains a mystery.
- Tom indicated an inclination to stick with that schedule for now and requested that anyone that will encounter attendance difficulties because of it let him know.
- Tom announced that the next meeting is scheduled for 2024-04-10 and that there are a number of papers awaiting review.

April 10th, 2024

Draft agenda:

- P2758R2: Emitting messages at compile time.

Attendees:

- Barry Revzin
- Corentin Jabot
- Fraser Gordon

- Jens Maurer
- Mark de Wever
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Due to a scheduling conflict, Barry was delayed in joining the meeting and review of P2758R2 was thus delayed. The time was filled with informal chat of various items including but not limited to:
  - Progress on P2873 (Remove Deprecated Locale Category Facets For Unicode from C++26).
  - The need, or lack thereof, for `u8streampos`, `u16streampos`, and `u32streampos`.
  - The Unicode Text Terminal Working Group.
  - U+FDFD (ARABIC LIGATURE BISMILLAH AR-RAHMAN AR-RAHEEM) and other characters with very wide display widths.
  - The past Tokyo and future St. Louis meetings.
  - Locales, `std::format()`, and `char8_t` support.
  - The Unicode Message Formatting Working Group.
  - ICU4X.
- P2758R2: Emitting messages at compile time:
  - Barry provided an introduction:
    - The goal is to allow programmers to produce more friendly diagnostics.
    - `static_assert` has limitations and clever hacks only go so far.
    - Producing errors is great, but there is value in being able to produce informational messages and warnings that can be elevated to errors.
    - `std::format()` is not declared `constexpr`, but probably could be.
    - The proposal is minimal and intended to provide infrastructure on which better interfaces can be built.
  - Victor posited that it would be useful to have a portable way to suppress a warning in a portable manner; a portable version of the `#pragma` directives that many implementations support today.
  - Victor stated that the paper needs updates to reflect the adoption of P2741R3 (user-generated `static_assert` messages).
  - Mark expressed support for the paper and commented that he recently asked Clang developers about such a feature.
  - Victor noted that clang-tidy allows a comment-based annotation to suppress diagnostics that emanate from specified source code lines.
  - Barry asked if such annotations would be expected to suppress diagnostics that would be produced from a specific call to one of these functions.
  - Victor replied affirmatively and stated that it would be difficult for his organization to enable these warnings otherwise without a way to suppress false positives.

- Jens explained that clang-tidy annotations are written at the line where the diagnostic is issued from and that the annotation Victor is interested in would have to work differently.
- Victor agreed and stated this suppression would be more complicated.
- Tom suggested it would probably have to be an annotation that suppresses any indicated warnings that emanate from within the constant evaluation of the annotated source line.
- Corentin opined that this paper doesn't need to address suppression of a diagnostic.
- Corentin noted that display of a diagnostic is within the purview of the implementor.
- Corentin asserted that, as long as there is a tag available, that implementors can provide a means to suppress it.
- Tom replied that a tag is specified for `constexpr_warning_str()`, but not for the other cases.
- Tom stated that, from an implementation stand point, he could see treating errors as discretionary errors that can be demoted to warnings.
- Barry replied that production of an error is intended to halt constant evaluation.
- Barry said that there are use cases for both fatal and discretionary errors, but that he doesn't really agree with motivation for the latter.
- Victor expressed opposition to being able to demote an error to a warning.
- Corentin observed that the wording needs to require that the message is provided in the ordinary literal encoding.
- Corentin reported that wording examples can be found in the wording for `static_assert`.
- *[Editor's note: see [dcl.pre]p12.]*
- Jens clarified that the elements of the `std::string_view` that holds the message will be considered code units of the ordinary literal encoding.
- Barry reported having located the wording and indicated he can copy it.
- Jens asked if `constexpr_error_str()` is equivalent to `static_assert(false, "message")`.
- Barry replied that it is very similar.
- Corentin explained that the evaluation is performed at a different time and potentially for a different number of occurrences; a `static_assert` will be evaluated once at translation or template instantiation time where as `constexpr_error_str()` may be evaluated multiple times during constant evaluation.
- Corentin asked what the expectations for a call to `constexpr_error_str()` are; for example, whether a diagnostic with different color highlighting would be produced.
- Corentin asserted that it should be possible to suppress each message kind; they should all have a tag for this reason.
- Corentin asked if escape sequences may appear in the message strings.
- Barry asked what `static_assert` does and was informed it is implementation-defined.

- *[Editor's note: examples with hilariously predictable implementation divergence can be seen at https://godbolt.org/z/xasvnMPre.]*
- Victor agreed with the suggestion to add a tag to `constexpr_print_str()`.
- Victor asked how ill-formed tags are handled.
- Tom replied that tags should be restricted to the basic literal character set.
- Corentin stated that implementations should escape non-printable characters and ill-formed code unit sequences in the diagnostics they produce.
- Tom asked for confirmation that text in the message that looks like a *universal-character-name* would not be treated as such.
- Corentin confirmed.
- Jens observed that the paper proposes a library facility but that he is uncertain that it is.
- Jens stated that [intro.compliance.general] would need an update.
- Jens noted that section was updated to address the requirement for the `#warning` and `#error` directives to produce a diagnostic message.
- Jens asked why it would be necessary to state that the program is ill-formed rather than that the expression is not a core constant expression.
- Jens explained that ill-formed means a diagnostic must be produced, but an implementation can do what it wants otherwise.
- Jens asked if specifying these as ill-formed requires an implementation to refuse to translate the program and noted that this is currently only required for `#error`.
- Tom asked Barry if the intent is to match `#error`.
- Barry expressed uncertainty.
- Jens advised reading [intro.compliance.general].
- Corentin stated that the characters permitted in tags needs to be clarified; quotes, semicolon, and other characters that have special meaning in command line shells should be prohibited.
- Tom pondered whether this should really be a core language facility.
- Tom suggested the tag should be required to be an unevaluated string to facilitate audits.
- Victor expressed a preference for the tag being a string literal.
- Corentin observed that requiring a string literal would require a core language feature.
- Barry replied that he would eventually like to expose this functionality with more `std::format()` like capabilities but doing so wouldn't be possible if this is specified as a language feature; at least not without expression aliases or some other way to pass a tag through a library interface.
- Tom stated he would like to review the proposal in SG16 again to review limitations on tags and wording for encoding requirements.
- Jens indicated that CWG will need to review the paper as well and stated he has a gut feeling that there is something missing.
- Jens noted that erroneous behavior is increasing motivation for producing something akin to diagnostics at run-time.

    - Jens suggested that LEWG might not have a lot of input since the library interface would just forward calls to a builtin function; that builtin function will require input from core implementors.
- Tom announced that the next meeting will be on 2024-04-24 and that he would work with authors to get papers scheduled with more advance notice this time.

April 24th, 2024

Draft agenda:

- P1953R0: Unicode Identifiers And Reflection.
- P2996R2: Reflection for C++26.

Attendees:

- Andrei Alexandrescu
- Braden Ganetsky
- Corentin Jabot
- Dan Katz
- Daveed Vandevoorde
- Eddie Nolan
- Giuseppe D'Angelo
- Jens Maurer
- Mark de Wever
- Nathan Owens
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Wyatt Childers

Meeting summary:

- P1953R0: Unicode Identifiers And Reflection:
    - Corentin provided an introduction:
        - This is an older paper and reflection has changed in the meantime, but it is still relevant.
        - P1949 (C++ Identifier Syntax using Unicode Standard Annex 31) clarified the syntax for identifiers to provide better support for non-English speakers and mathematicians.
        - String literals are converted from the source file encoding to an implementation-defined literal encoding that might not be Unicode.

        - P1854R4 (Making non-encodable string literals ill-formed) changed string literals to be ill-formed if they specify characters that are not representable in the associated literal encoding.
        - Characters can always be converted to Unicode encodings without loss of data in C++.
        - Reflection needs to specify the type and encoding used to reflect an identifier.
        - The only solution that works in all cases is to expose identifiers in a UTF encoding.
        - It is not possible to infer the encoding of a string just by looking at the values of its code units.
    - Daveed commented that there are some encodings that have characters that lack representation in Unicode.
    - Corentin acknowledged such limitations and explained that new characters are regularly invented in some cultures but are not widely used or encoded.
    - Corentin noted that trademarks and various other symbols likewise are not encoded in Unicode.
    - *[Editor's note: The editor's Bluetooth stack regretably crashed and a minute or so of Corentin's continued elaborations were not captured.]*
    - Victor stated that having reflection expose names solely in `char8_t` would be user hostile since there is little support for `char8_t` in the standard library.
    - Victor reported that his organization bans use of `u8` literals.
    - Victor expressed support for the approach described in section 4.4.6, "`name_of`, `display_name_of`, `source_location_of`" that limits names to characters in the *basic character set*.
    - Victor asserted that reflection must provide good support for the common case where the ordinary literal encoding is UTF-8.
    - Daveed asked about implications for EBCDIC based platforms.
    - Corentin replied that EBCDIC and UTF-8 encoded data can't be discerned just by looking at the string contents, so reflecting names in UTF-8 in `char`-based storage would be problematic for such platforms.
    - Tom replied that there are EBCDIC code pages that are missing representation for some characters from the basic character set but that digraphs are available for those characters so we don't really concern ourselves with them in practice.
    - Steve corrected Tom in the chat; EBCDIC code pages provide representation for all the characters in the basic character set, but not all such characters are encoded with the same value.
    - Jens explained that, for the purpose of this discussion, it is important to recognize that EBCDIC and ASCII map characters of the basic character set to different code points and are therefore not compatible.
- P2996R2: Reflection for C++26:
    - Daveed presented:
        - *[Editor's note: Daveed's presentation slides are avilable here.]*

        - An overview of the proposed reflection syntax was provided.
        - There are three functions that reflect the names of entities at present, but more might be added.
        - There is only one function that consumes names as strings right now, but more might be added.
        - Names provided by some reflection interfaces must be consumable in the same form by other reflection interfaces.
        - The ability to write names to `std::cout` is required.
        - It is ok for the names to not be source-like; `std::meta::display_name_of()` can use a descriptive notation.
        - The translation model is Unicode based so names can be provided in Unicode encodings, but the standard library is missing support for text in `char8_t`.
        - Proposal sketch #1:
            - Provide names in both `char` and `char8_t` based storage and associated encodings.
            - Require names to round-trip.
        - Proposal sketch #2:
            - Provide names only in `char8_t` based storage and UTF-8; names naturally round-trip.
            - Make `std::cout` work with UTF-8 text in `char8_t`.
        - *[Editor's note: Proposal sketch #3 in the linked slides was added after the meeting as inspired by ensuing discussion.]*
    - Jens asked if `name_of()` is proposed as a `consteval` function.
    - Daveed confirmed that it is.
    - Tom asked for clarification regarding the intended use cases for `name_of()`, `qualified_name_of()`, and `display_name_of()`.
    - Daveed replied that `name_of()` is intended to return an identifier or a canonical name such as `operator X` and that `qualified_name_of()` and `display_name_of()` are intended to return potentially localized descriptive text.
    - Andrei observed that programmers might want to pass a `data_member_options_t` object around and that the `optional<string_view> name` member is potentially problematic for lifetime reasons.
    - Daveed acknowledged that the data member type might need to be changed to an owning string type.
    - Corentin explained that conversion from an arbitrary encoding to Unicode might not roundtrip because characters like Å (U+212B ANGSTROM SIGN) and Å (U+00C5 LATIN CAPITAL LETTER A WITH RING ABOVE) are distinct in Unicode, but might not be distinct in the ordinary literal encoding.
    - *[Editor's note: The Å (U+00C5 LATIN CAPITAL LETTER A WITH RING ABOVE), Å (U+212B ANGSTROM SIGN), A (U+0041 LATIN CAPITAL LETTER A), and ˚(U+030A COMBINING RING ABOVE) characters are all individually permitted in Unicode identifiers. However, since C++ identifiers are required to be in Unicode normalization form C (NFC), only the first form (U+00C5) is permitted in a C++*

identifier. The ordinary literal encoding is not restricted to NFC, so this character could be converted to one of the other forms and therefore fail to round-trip. This could result in a requirement for implementations to perform conversion to NFC when consuming names. See the "Singleton Exclusions" section of UAX #15 (Unicode Normalization Forms).]

- Steve asked if there is a desire or requirement to be able to emit text containing names at compile-time.
- Daveed responded negatively and stated that the `std::cout` requirement is intended as a debugging aid.
- Corentin responded to Victor's earlier statements regarding lack of support for `char8_t` in the standard library and asserted that we should fix that.
- Corentin expressed support for providing names in both `char` and `char8_t`.
- Corentin stated that reflection is an important feature and that we shouldn't implement hacks just to workaround the missing support for `char8_t`.
- Corentin insisted that improving support for `char8_t` is a tractable problem and that we have some time for improvements in C++26.
- Victor opined that reflection should not be dependent on `std::string_view`.
- Victor stated that it took a long time to properly specify `std::print()` and that we shouldn't implement hacks in iostreams just to make `std::cout` work with `char8_t` in the C++26 timeframe.
- Victor explained that the model we are moving towards is one where the ordinary literal encoding is UTF-8.
- Victor suggested that an identifier or name type could be provided instead of a string; this would enable writing formatters for it.
- Eddie asked Corentin if there are round-trip normalization concerns and whether renormalization is required.
- Corentin replied negatively and stated that there are characters that are duplicated in Unicode and do not normalize to each other.
- Eddie replied that identifiers are required to be in NFC.
- Tom stated that we are not going to be able to reach a conclusion on round-tripping and renormalization now and that we'll need to research and revisit.
- Tom said he is not convinced that normalization is a significant issue.
- Daveed asked what the deadline is for new library feature proposals for C++26.
- Jens provided a link to P1000R5 (C++ IS schedule) and reported that the Wrocław meeting in November is the last meeting for core language features that require a response from LEWG and that the Hagenberg meeting in February is the last meeting to forward papers to CWG and LWG.
- Tom expressed support for Victor's suggestion of a distinct formattable type for names and identifiers.
- Tom agreed with Victor regarding optimizing for the case where UTF-8 is the ordinary literal encoding, but disagreed with the suggestion that `char` will ever imply UTF everywhere.

- Tom expressed a preference for exposing names in both `char` and `char8_t` based storage.
- Daveed described limitations of constant evaluation that make use of `std::string` problematic, but noted that implementations can provide views backed by data in a string literal pool.
- Corentin noted that the encoding challenges remain the same if a unique type is used; a solution is still needed to enable printing of it.
- Corentin acknowledged that an opaque type might confer other benefits.
- Jens agreed with Tom that, while we might like for UTF-8 to take over everywhere, environments that rely on EBCDIC are likely to remain.
- Jens asserted that we must take backward compatibility into account.
- Jens observed that there are two levels of encoding:
  - At compile-time, data might or might not be UTF-8, but the encoding is known if a name is produced and consumed during constant evaluation.
  - At run-time, the encoding of the environment might be different and might require transcoding or some form of escaping to not lose data.
- Jens noted that we explicitly decided not to interfere with the existing behavior of `std::cout` and introduced `std::print()` as a new interface.
- Jens asked how programmers will produce new names based on reflected ones given that `std::format()` is not declared `constexpr`.
- Jens expressed uncertainty regarding what locale means during constant evaluation.
- Jens suggested that returning an opaque type might be useful, but is also not so different from returning `std::string_view` and providing additional library support.
- Daveed stated that the addition of a distinct type creates some complexity but that it could be associated with statically allocated memory.
- Daveed noted that the creation of lots of names could produce massive numbers of string literals if names are backed by string pools and stated there could be an advantage to the distinct type approach.
- Eddie observed that an opaque type that converts to both `std::string_view` and `std::u8string_view` could result in ambiguous conversions for formatted printing.
- Dan observed that an opaque type helps to make it clear to the user that they might want to perform some operations on it before printing it.
- Corentin responded to Eddie's observation by stating that, as long as the opaque type doesn't require conversion in order to be printed, then there are no ambiguous conversion concerns.
- Corentin observed that SG16 talks about EBCDIC a lot, but noted that Windows is not UTF-8 by default and that Shift-JIS is still the main encoding used in Japan.
- Corentin agreed with Victor that it would be nice to have `char` be synonomous with UTF-8 but stated that isn't the world we live in.
- Corentin noted that, when writing output to a terminal, we can't guarantee that an identifier can be accurately displayed due to encoding limitations, encoding conversion limitations, and fonts.

- Corentin stated that `std::format()` and `std::print()` do a much better job than iostreams and that `std::print()` will print Unicode correctly on Windows; that can't be fixed for iostreams.
- Corentin asked Daveed if non-transient memory allocation is still being pursued.
- Daveed responded that it probably is not feasible to deliver in C++26.
- Victor also responded to Eddie's observation by opining that he doesn't think implicit conversions from an opaque type would be an issue for `std::format()` but that he wasn't sure about iostreams.
- Victor noted that writing `char8_t` to iostreams will be lossy or produce mojibake.
- Victor stated that `constexpr` support for `std::format()` is frequently requested and asserted that we should prioritize that over adding new support for `char8_t`.
- Victor reported that proposals for compile-time messages have expressed interest in `constexpr` support for `std::format()`.
- Tom posted the following candidate polls in the chat:
  - Candidate poll 1: P2996R2: identifier names should be made available via `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` consistent with `std::filesystem::path` and [fs.path.native.obs].
  - Candidate poll 2: P2996R2: identifier names returned by `name_of()` in `char`-based storage should be encoded in the ordinary literal encoding with non-representable characters rendering the call ill-formed.
  - Candidate poll 3: P2996R2: identifier names returned by `display_name_of()` in `char`-based storage should be encoded in the ordinary literal encoding with non-representable characters escaped as in [format.string.escaped].
  - Candidate poll 4: P2996R2: `char`-based identifier names accepted by `data_member_spec()` (via `data_member_options_t`) should be encoded in the ordinary literal encoding.
- Corentin expressed concern about memory footprint if names are backed by string literals and made available in multiple encodings.
- Daveed responded that the strings are only generated when you actually use them; Victor's opaque type would effectively have a handle to an internal representation backed by static storage.
- Tom pointed out that conversions from the internal representation could then be performed at run-time.
- Victor expressed curiosity about candidate poll 1.
- Tom explained the thoughts that motivated that poll suggestion; `std::filesystem::path` provides a precedent for providing conversions to various encodings; if this poll has consensus, then there is no need to poll support for individual encodings; if not, we can.
- Tom posted the following alternatives to candidate poll 1 in the chat:
  - Candidate poll 1.1: P2996R2: identifier names should be made available in `char`-based storage.
  - Candidate poll 1.2: P2996R2: identifier names should be made available in `char8_t`-based storage.

        - Candidate poll 1.3: P2996R2: identifier names should be made available in `char16_t`-based storage.
        - Candidate poll 1.4: P2996R2: identifier names should be made available in `char32_t`-based storage.
        - Candidate poll 1.5: P2996R2: identifier names should be made available in `wchar_t`-based storage.
    - Victor expressed support for candidate poll 2, noted that we didn't discuss it yet, but likes that it enables support for all possible identifiers in UTF-8 in `char`-based interfaces when the ordinary literal encoding is UTF-8.
    - Steve noted that there is the possibility of problems caused by translation units being compiled with different ordinary literal encodings.
    - Steve suggested that it might be useful to provide a library interface that can produce strings with UCN-like sequences substituted.
    - Steve noted that use of an opaque type would enable use with any of the range encoding libraries.
    - Daveed stated that the P2996 authors would be opposed to support for all five character types but that they are ok with support for `char` and `char8_t`.
    - Tom asked for clarification regarding opposition for support of the other character types.
    - Daveed responded that common storage can be used to back the same representation for `char` and `char8_t`, but that isn't the case for the other character types.
    - Corentin noted that `char16_t` and `char32_t` are also less efficient to store.
    - Corentin stated that he is not opposed to an opaque type as long as it can be printed as Unicode with good results.
    - Corentin asserted that we still need to make `char8_t` work in the standard library regardless.
    - Corentin expressed opposition to introduction of an escape mechanism that effectively introduces an additional encoding.
    - Corentin suggested that if we want to support `wchar_t`, `char16_t`, and `char32_t`, that we should provide a translation interface rather than duplicating interfaces throughout the standard library.
    - Daveed responded with "Amen, brother!"
    - Eddie stated that P2728 (Unicode in the Library, Part 1: UTF Transcoding) is fully constexpr and would provide support for conversion to UTF-16 in `char16_t` and UTF-32 in `char32_t`.
- Tom requested that Daveed make his presentation available for inclusion in the meeting summary.
- Daveed immediately obliged.
- Tom announced that the next meeting will be held May 8th and that we'll continue discussion of this paper then.
- Tom apologized to Corentin and lamented that this will once again delay further review of P2626 (`charN_t` incremental adoption: Casting pointers of UTF character types).

May 8th, 2024

Draft agenda:

- D3258R0: Formatting of charN_t.
- P2996R2: Reflection for C++26.

Attendees:

- Braden Ganetsky
- Corentin Jabot
- Dan Katz
- Eddie Nolan
- Lauri Vasama
- Mark de Wever
- Nathan Owen
- Peter Bindels
- Robin Leroy
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Robin provided a report on UTC #179:
  - *[Editor's note: Minutes from the UTC #179 meeting are recorded in L2/24-061.]*
  - The alpha review period closed several weeks before the meeting and the UTC WGs then had one week to prepare any material responses for the meeting.
  - The agenda for this meeting included reviewing the alpha feedback and authorizing the beta release with stable specifications.
  - The character repertoire is now frozen.
  - Two recently added characters were removed at the request of the Indian government; see consensus item 179-C43.
  - Significant changes were made to the line breaking algorithm, but these changes don't affect current C++.
    - Improvements were made to the handling of quotation marks in simplified Chinese.
    - Lines are no longer broken after hyphens that separate Hebrew and non-Hebrew text.
  - Recommendations from the CJK & Unihan Working Group were accepted that will impact the wording currently present in [format.string.std]p13 when the C++ standard is rebased on Unicode 16; the set of code points included in bullet 13.2

will be subsumed by 13.1 due to acceptance of L2/24-059 (Proposal to change the East_Asian_Width property of the Yijing symbols).
    - Tom stated that we should create an issue to track doing that update when we rebase on Unicode 16 or later.
    - *[Editor's note: Tom created SG16 issue 81 (Unicode 16: Updates needed for [format.string.std]p13 field widths) to do so.]*
- Robin wondered why the code points listed in [format.string.std]p13 bullets 13.3 (U+1f300 - U+1f5ff (Miscellaneous Symbols and Pictographs)) and 13.4 (U+1f900 - U+1f9ff (Supplemental Symbols and Pictographs) are listed with a field width of 2; these code points aren't wide in text presentation form, but would be in emoji presentation form.
- Robin shared a link listing all of the characters covered by bullet 13.3 and noted that some of them, 𝌆 for instance, are presented in a narrow form in the Windows terminal for him.
- Corentin explained that testing revealed that these characters were predmoninantly displayed as wide characters in existing terminals.
- Eddie reported relevant discussion having occured during the recent meeting of the Unicode Text Terminal Working Group (TTWG); the POSIX `wcswidth()` function maps a code point to a width, but does not account for variation selectors.
- Eddie stated that there is supposed to be a default for whether text vs emoji presentation form is used, but there is implementation divergence.
- D3258R0: Formatting of charN_t:
    - *[Editor's note: D3258R0 was the active paper under discussion at the telecon. The agenda and links used here reference P3258R0 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
    - Corentin provided an overview of the paper:
        - The motivation for the paper is to enable the ability to print `char8_t`-based UTF-8 text via `std::format()`.
        - The intent is for something like `std::format("...", std::meta::name_of(^XX))` to just do the right thing.
        - The goal is for semantics to be consistent.
        - The proposal includes support for formatting arguments of type `char8_t`, `char16_t`, and `char32_t` for both `""` and `L""` format strings.
        - No support is proposed for use of `u8""`, `u""`, or `U""` literals as format strings.
        - A replacement character will be substituted for ill-formed code unit sequences.
        - No error mechanism is proposed but one could be added later by adding format specifier options.
        - No support is proposed for formatting arguments of type `char` with a `L""` format string or for formatting arguments of type `wchar_t` with a `""` format string due to potentially ambiguous encoding associations.
        - Formatting of escaping characters and strings will work as expected.

        - For a non-UTF encoding, the replacement character will be `?`; this matches substitutions currently observable with the Microsoft compiler on Windows.
        - No special behavior is proposed for `std::print()`.
        - A prototype implementation was completed for libc++, but libc++ only supports the ordinary literal encoding being UTF-8, so that doesn't exercise transcoding scenarios.
        - The C and C++ standards don't provide transcoding facilities other than `mbrtoc8()` and such, but conversions can be done using `iconv`, ICU, or other existing converters.
        - Some transcoding facilities do not offer flexibility for error handling.
        - Support for formatting single code units of `char8_t`, `char16_t`, and `char32_t` is proposed; this is consistent with existing support for `char` and `wchar_t`.
        - `constexpr` implementations of `std::format()` already have the ability to perform conversions between the set of literal encodings.
    - Victor observed that a number of the `std::format()` examples in the paper are syntactically incorrect as presented; likely due to markup issues.
    - Victor explained that `std::vprint_unicode()` and `std::vprint_nonunicode()` are not exposition only so that programmers can provide overloads for their own types with differentiation for UTF and non-UTF encodings.
    - Victor noted that locking variations of these functions are now specified as well.
    - *[Editor's note: Locking variations were recently added via the adoption of P3107R5 (Permit an efficient implementation of std::print) during the Tokyo meeting.]*
    - Tom asked for an explanation of the ABI limitations on extending format specifiers.
    - Mark explained that the ABI is restricted by `std::basic_format_arg<Context>::visit()`; `std::basic_format_arg` is effectively a discriminated union and the number of discernible types is constrained by the type used to identify them.
    - *[Editor's note: See Mark's follow up post to the SG16 mailing list.]*
    - Victor replied that it would be possible to use the normal formatter API instead of `std::basic_format_arg`.
    - Victor stated that {fmt} already supports `constexpr`, but that there are no immediate plans to propose support for `std::format()` as `constexpr` in the standard.
    - Victor suggested that the paper simply state that `constexpr` support is implementable.
    - Tom directed discussion to whether the proposed capabilities would suffice to meet the minimum requirements for printing of identifiers as desired for the reflection proposal.
    - Dan opined that it does, noted that Daveed would like to have iostream support, but commented that he doesn't feel as strongly about that.
    - Corentin said that he would like to know if anyone felt very strongly about support for iostreams.

    - Corentin stated he would rather focus on support for `std::format()` and `std::print()`, but that he can understand why others might want iostream support specifically.
    - Corentin explained that he did not propose iostream support because he didn't feel like he was the right person to do so.
    - Victor stated that he views the proposed capabilities as a partial solution that is not inline with the `std::format()` design intent to not mix encoding concerns.
- P2996R2: Reflection for C++26:
    - Victor expressed strong opposition to only exposing identifiers in `char8_t` and asserted that we need to figure out the story for support of `char`.
    - Dan interpreted Victor's response as meaning that the proposed facilities with only support for `char8_t` does not provide a sufficient solution.
    - Dan stated that the idea of using a magic proxy type seems good and that Daveed has expressed support for it.
    - Eddie reported having recently discussed the proxy type with other attendees of C++Now and that some found it to be an overcomplicated solution.
    - Corentin responded that, if done right, programmers shouldn't be much affected by it.
    - Corentin opined that a proxy type is fine but that a solution that uses an escape mechanism to effectively create a new encoding is not.
    - Tom pondered whether there is a need to distinguish between names and identifiers and noted that many functions, like conversion operators and overloaded operators, don't have associated identifiers but do have names.
    - Corentin indicated that is probably not an SG16 concern.
    - Corentin suggested that reflection use cases are best addressed by performing code injection rather than defining overloaded operators.
    - Corentin agreed that reflection might want to differentiate names and identifiers in a similar manner to how Clang does internally.
    - Dan advised caution regarding exposing a meta type for a name when already working with a meta type.
    - Victor asked if the hypothetical proxy name type might be exposition only with conversion operators.
    - Victor asked what the plan is for exploring the idea of such a type.
    - Corentin asked with a smile whether Victor was volunteering to do so.
    - Dan responded that the P2996 authors can propose a shape for the type.
    - Tom summarized his impression of consensus so far; that it seems that there is good consensus for supporting both `char` and `char8_t`, but since we can't overload based on return types, that use of a distinct type is needed to avoid having to specify distinct names; such a type enables future extension.
    - Corentin asked what the motivation would be for an exposition only type.
    - Dan replied with a smile that it would avoid a bike shedding exercise in LEWG.
    - Dan stated there is a need to be able to perform string comparisons for implementation of `enum_to_string`.

- Corentin acknowledged that a proxy type makes things easier by adding a layer of indirection.
- Eddie asked for reasons not to define separate functions for `std::meta::name_of()` and related functions.
- Dan replied that doing so creates a combinatorial explosion.
- Eddie asked what would happen in the case of an enumeration that has a set of enumerators that cannot all be converted losslessly to the ordinary literal encoding and where transliteration might produce the same name.
- Dan suggested use of `char8_t` to avoid such cases.
- Eddie responded that his concern is whether such a scenario should be possible since it makes it easy to do the wrong thing.
- Corentin noted that the compiler's internal representation is always able to distinguish such cases.
- Corentin asked Robin if there are duplicated characters that are valid for use in identifiers and that canonicalize to the same representation.
- Robin replied that there are reasonable mappings to other character sets that result in ambiguity.
- Robin provided a reference and some examples in the chat:
  - Unicode 15.1, chapter 7, section 7.2, "Greek", paragraph starting with "Greek Letters as Symbols":

    > For compatibility purposes, a few Greek letters are separately encoded as symbols in other character blocks. Examples include U+00B5 µ MICRO SIGN in the Latin-1 Supplement character block and U+2126 Ω OHM SIGN in the Letterlike Symbols character block. The ohm sign is canonically equivalent to the capital omega, and normalization would remove any distinction. Its use is therefore discouraged in favor of capital omega. The same equivalence does not exist between micro sign and mu, and use of either character as a micro sign is common. For Greek text, only the mu should be used.

    - µ, μ, **μ**, *μ*, ***μ***, **μ**, and ***μ*** are all compatibility equivalent to μ and all are valid C++ identifiers.
- Corentin asked whether the roundtrip requirement can actually be satisfied in the presence of arbitrary encodings.
- Victor replied to the overloading concerns by mentioning that Daveed's original suggestion was for `std::meta::name_of()` and friends to be templated on a character type.
- Victor noted that roundtrip support can be facilitated with an escape mechanism as in Daveed's preferred option.
- Tom stated that roundtrip support cannot tolerate lossy conversions and that an attempted conversion that would be lossy must result in an error or substitution of an escape sequence.
- Eddie expressed concern that conversion to `char` won't work everywhere, but since it will work in most cases such support can lead to broken code that isn't caught

by testing.
    - Eddie suggested that the function template idea seems ok if the character template type parameter is specified with a default template argument of `char8_t`.
    - Dan asked what the advantage of a template parameter would be over an opaque type.
    - Eddie replied that it can't be completely hidden away; that it will appear in error messages, on cppreference.com, etc...
    - Corentin stated that programmers don't want to care about this and that they just want the identifier to be printed; if we can make it just work, that is a win.
- Tom announced that the next SG16 meeting will be on 2020-05-22 and that he intends to put P2626R0 (charN_t incremental adoption: Casting pointers of UTF character types) on the agenda, perhaps along with some recently created LWG issues.